

Java 8

Concrete Methods in Interfaces

In Java 8, interfaces can have concrete methods using the default keyword. Before Java 8, all methods in an interface had to be abstract, so every class had to write its own code. This was a problem if we wanted to add new methods to an old interface, because all existing classes would break. To fix this, Java 8 introduced default methods.

1. Using Default Methods

A **default method** already has a body inside the interface, so classes that implement the interface can use it directly without writing extra code. If a class wants, it can also override the default method and give its own version.

2. Using Static Method

Java 8 also introduced static methods inside interfaces. These belong to the interface itself, not to objects. They must be called using the interface name. They cannot call default or private instance methods, because they don't run on objects. They can call private static methods and access constants.

```
package com.java8;

interface VTU_MYS{
    int branches = 10;

    default void cse_fee() {
        System.out.println("Branches: "+ branches);
        System.out.println("50K");
        //can access variables
        //can call private method and private static method
        loc(); //can call static method
        data(); //can call private method
        drives(); //can call private static method
    }

    static void loc() {
        System.out.println("Mys branch: "+ branches);
        //can access variable
        //but can not call default method
    }

    //    cse_fee(); //Throws CTE
    //    data();    //Throws CTE
    drives();    //can call private static method
}

private void data() {
    //can access variables in interface
    System.out.println("Branches: "+ branches);
    System.out.println("2023, 50k students joined");
}

private static void drives() {
    //can access variables in interface
    System.out.println("Branches: "+ branches);
    System.out.println("2023, 50 drives");
}
}
```

3. Private Methods

Java 9 introduced private methods in interfaces. These are helper methods used only inside the interface itself. Private instance methods: used by default methods. Private static methods: used by static methods (and also by default methods). They cannot be called by implementing classes or from outside.

```
class Mys implements VTU_MYS{
}

public class Vtu2 {

    public static void main(String[] args) {
        VTU_MYS vm = new Mys();
        vm.cse_fee();
        // vm.loc(); //using reference variable only default methods can be called
        // vm.data();
        // vm.drives();

        VTU_MYS.loc();
        // VTU_MYS.cse_fee(); //using interface name we can only call static methods
        // VTU_MYS.data();
        // VTU_MYS.drives();

        //private methods can only be called in default method and private static method
        //in static method and default method
    }
}
```

Lambda Expression

lambda expression is a way to write short, inline functions without creating a separate method or class. It provides functional programming style in Java. Mainly used to implement functional interfaces (an interface with exactly one abstract method, e.g., Runnable, Comparator, Callable).

```
package com.java8;

//LAMBDA EXPRESSION

//the method must be declared in the interface

//Less lines of code
//can provide implementation for functional interface
//do not need to define method again to provide implementation
//can be used to iterate, filter and extract data from collections
//is an anonymous function - no name, modifier and return type

//when u have a method like:
//public void fee() {
//    System.out.print("50");
//}

//it can be written in lambda expression as

//()->{System.out.print("50")};
//if its a single statement inside {} then it can also be omitted
//()->System.out.print("50");

//for passing parameters:

//public void add(int a, int b) {
//    System.out.print(a+b);
//}

//it can be written in lambda expression as

//(int a, int b)->{System.out.print(a + b)};
//if its a single statement inside {} then it can also be omitted and datatype is also optional
//(a, b)->System.out.print(a+b);
```

```

interface VTU2{
    void fees();
}

public class Lambda //implements VTU2
{
    // @Override
    // public void fees() {
    //     // TODO Auto-generated method stub
    // }

    public static void main(String[] args) {
        VTU2 v = ()->System.out.println("Fees is 50k"); //only for defining the implementation
        v.fees(); //calling the method that is implemented by lambda expression
        //it can only be called with reference of the interface

        Lambda l = new Lambda();
        // l.fees(); //gives CTE

        VTU2 v2 = new Lambda();
        // v2.fees(); method must be implemented in the class normally to get rid of both the errors
    }
}

```

Functional Interface

A Functional Interface is an interface with exactly one abstract method. It may contain default methods and static methods, but only one abstract method. Marked with the [@FunctionalInterface](#) annotation (optional, but recommended).

Examples from Java: Runnable, Callable, Comparator, Consumer, etc.

Relation to Lambda Expressions - Lambda expressions provide the implementation of the single abstract method of a functional interface. That's why functional interfaces are also called SAM interfaces (Single Abstract Method).

```

package com.java8;

@FunctionalInterface
interface Names{
    public void display(String name);
    // void add();
    //can not be written as it is only for functional interface that is- interface with single method declaration
}

public class Lamb2 {
    public static void main(String[] args) {
        Names n = (name)->{System.out.println("Hi! "+name)};
        n.display("Leo");
    }
}

```

Traditional Approach (Before Java 8)

Normally, we create a class that implements the interface and overrides its abstract method.

Lambda Expression Approach (Java 8)

Instead of creating a separate class, we can use a lambda expression directly to provide implementation.

Example 1:

```
package com.java8;

interface VTU11{
    public double fee(double fe);
}
interface VTU12{
    public double fee(double pt, double govt);
}
public class Lambda3 implements VTU11{
    @Override
    public double fee(double fe) {
        return fe;
    }
    public static void main(String[] args) {
        //traditional approach:

        VTU11 v = new Lambda3();
        System.out.println(v.fee(2000));

        //new approach using lambda expression

        VTU11 v1 = (fe)-> fe + 100;

        VTU11 v2 = (f)->{return f+200;};

        System.out.println(v1.fee(2000));
        System.out.println(v2.fee(2000));

        //using 2 parameters:
        VTU12 v3 = (pt, gvt)->{
            return pt+gvt;
        };

        System.out.println(v3.fee(200, 300));
    }
}
```

Method Reference in Java 8

Method Reference (MR) is a shorthand notation of a lambda expression that directly calls an existing method.

Syntax:

ClassName::methodName

objectRef::methodName

It is used when a **lambda expression only calls an existing method**. Makes code shorter and more readable.

Functional Interface Requirement

Method reference works only with functional interfaces (SAM – Single Abstract Method).

```
package com.java8;

//MR-> reference to one method from another m
@FunctionalInterface
interface VTU4{
    public void fees();
}

public class MethodReference {
    public static void Loc() {
        System.out.println("Mysore");
    }

    public static void main(String[] args) {
        VTU4 v = MethodReference::Loc;
        v.fees(); //fees() invokes location-
    }
}
```

VTU4 is a functional interface with fees(). MethodReference::Loc means → **reference to static method Loc()**. When we call v.fees(), it internally calls Loc() method.

Example

```
import java.util.function.BiFunction;

interface java{
    void say_yes(int a);
}

class College3{
    College3(){
        System.out.println("welcome to java development");
    }
}

interface oracle{
    void get_offer();
}

class College2{
    public void say_yes_to_development() {
        System.out.println("Hello, kindly read well...");
    }
    public void say_yes_to_development(int a) {
        System.out.println("Hello, kindly read well..." + a);
    }
}

class College{
    public static int fee(int c_fee, int p_fee) {
        return (c_fee + p_fee);
    }
}

public class Example1 {
    public static void main(String[] args) {
        BiFunction<Integer, Integer, Integer> f = College::fee;
        System.out.println(f.apply(10000, 40000));

        College2 c = new College2();

        java j = c::say_yes_to_development;
        j.say_yes(5);

        java j1 = new College2()::say_yes_to_development;
        j1.say_yes(5);

        oracle o = College3::new;
        o.get_offer();
    }
}
```

- Program execution starts in the main method.
- A BiFunction is created referencing the static method College::fee.

- f.apply(10000, 40000) is called, printing 50000.
- A College2 object c is created.
- An instance method reference c::say_yes_to_development is assigned to java j.
- j.say_yes(5) is invoked, printing "Hello, kindly read well...5".
- Another instance method reference new College2()::say_yes_to_development is assigned to java j1.
- j1.say_yes(5) is invoked, printing "Hello, kindly read well...5".
- A constructor reference College3::new is assigned to oracle o.
- o.get_offer() is invoked, creating a new College3 object and printing "welcome to java development".
- Program execution ends.

Predefined Functional Interfaces in Java 8

Java 8 provides many built-in functional interfaces in the package java.util.function.

The most commonly used are:

1. Predicate
2. BiPredicate
3. Consumer
4. BiConsumer
5. Supplier
6. Function
7. BiFunction

1. Predicate<T>

It represents a function that takes one input and returns a boolean.

Method: boolean test(T t). **Use case:** Filtering data, conditions

```
package com.java8;

import java.util.function.Predicate;

//Predicate
//returns boolean
//has only one method - test()

public class PredefinedFI {

    public static void main(String[] args) {
        Predicate<Integer> p = (i) -> i>10;    //condition which is checked

        System.out.println(p.test(20));
        System.out.println(p.test(2));
    }
}
```

```

package com.java8;

import java.util.function.Predicate;

public class Predicate2 {

    public static void main(String[] args) {
        String[] names = {"Rutu", "Sana", "Sam"};
        Predicate<String> p = (name) -> name.charAt(0) == 'k';
        //char at index 0 == k ie name starts with k

        for(String s: names) {
            p.test(s);    //tests if the first letter on the name starts with k
            System.out.println(s);
        }
    }
}

```

Multiple predicates can be combined using:

- and() → both conditions must be true.
- or() → at least one condition must be true.
- negate() → reverses the result.

Problem Statement

Display employees who are working in Development role AND are located in Bangalore.

```

import java.util.Arrays;

//multiple predicates can be joined using and() or or()

//Display the name of the emp who is working in development and is working in Bangalore

class TCS{
    String name;
    String id;
    String loc;
    String role;
    public TCS(String name, String id, String loc, String role) {
        super();
        this.name = name;
        this.id = id;
        loc = loc;
        this.role = role;
    }
}

public class PredicateJoining {

    public static void main(String[] args) {
        TCS t1 = new TCS("Rutu", "TCS001", "Banglore", "Dev");
        TCS t2 = new TCS("Renu", "TCS002", "Banglore", "Dev");
        TCS t3 = new TCS("Sana", "TCS003", "Mysore", "Bio");
        TCS t4 = new TCS("Ram", "TCS004", "Banglore", "Gold");

        List<TCS> t = Arrays.asList(t1, t2, t3, t4);

        Predicate<TCS> p1 = (i) -> i.loc.equals("Banglore");
        Predicate<TCS> p2 = (i) -> i.role.equals("Dev");

        Predicate<TCS> p = p1.and(p2);

        for(TCS emp: t) {
            if(p.test(emp)) {
                System.out.println(emp.name + " - " + emp.loc + " - " + emp.role);
            }
        }
    }
}

```

Key Points

- Predicate Joining allows us to write clean and reusable conditions.
- Useful in filtering collections with multiple criteria.
- `and()` → all conditions must be true.
- `or()` → any one condition is enough.
- `negate()` → reverses the condition.

2. BiPredicate<T, U>

Takes two inputs and returns a boolean.

Method: `boolean test(T t, U u)`. **Use case:** Compare or validate two values.

Problem Statement

Check if an employee is located in Bangalore AND has a salary greater than 50,000 using BiPredicate.

```
package com.java8;

import java.util.*;
import java.util.function.BiPredicate;

class Employee {
    String name;
    String location;
    double salary;

    public Employee(String name, String location, double salary) {
        this.name = name;
        this.location = location;
        this.salary = salary;
    }
}

public class BiPredicateExample {
    public static void main(String[] args) {
        Employee e1 = new Employee("Rutu", "Banglore", 60000);
        Employee e2 = new Employee("Renu", "Banglore", 45000);
        Employee e3 = new Employee("Sana", "Mysore", 70000);

        List<Employee> employees = Arrays.asList(e1, e2, e3);

        // BiPredicate with 2 inputs: location & salary
        BiPredicate<String, Double> isEligible =
            (loc, sal) -> loc.equals("Banglore") && sal > 50000;

        for (Employee emp : employees) {
            if (isEligible.test(emp.location, emp.salary)) {
                System.out.println(emp.name + " - " + emp.location + " - " + emp.salary);
            }
        }
    }
}
```


Key Points

- Predicate<T> → works on one input.
- BiPredicate<T, U> → works on two inputs.
- We can also combine BiPredicates using:
 - and() → both conditions must be true.
 - or() → at least one condition must be true.
 - negate() → reverses the result.

Example:

```
BiPredicate<String, Double> p1 = (loc, sal) -> loc.equals("Banglore");
```

```
BiPredicate<String, Double> p2 = (loc, sal) -> sal > 50000;
```

```
BiPredicate<String, Double> combined = p1.and(p2);
```

3. Consumer<T>

Takes **one input** and returns **nothing (void)**.

Method: void accept(T t). **Use case:** Printing, logging, performing actions.

```
package com.java8;

import java.util.Arrays;

//predefined consumer interface
//has only accept(T t) method
//will accept input but does not return value
//forEach(Consumer consumer) will take consumer as parameter

public class ConsumerFI {

    public static void main(String[] args) {
        Consumer<String> emp = (e) -> System.out.println("Hi " + e + "Welcome");
        emp.accept("Rutu");
        emp.accept("Bacca");

        List<Integer> list = Arrays.asList(1, 4, 3, 2);
        list.forEach(n -> System.out.println(n));
        list.forEach(System.out::println);
    }
}
```

Key Points

- **Consumer** → input accepted, but no return value.
- Commonly used with forEach() in streams and collections.
- Can be combined using:
 - andThen() → chain multiple Consumers.

Example:

```
Consumer<String> c1 = s -> System.out.print("Hello " + s);  
Consumer<String> c2 = s -> System.out.println(", welcome!");  
c1.andThen(c2).accept("Rutu");
```

Output:

Hello Rutu, welcome!

4. BiConsumer<T, U>

- BiConsumer<T, U> is a predefined functional interface in java.util.function.
- It represents an operation that:
 - Takes two inputs
 - Returns no result (void)
- Abstract method: void accept(T t, U u);

Example

```
package com.java8;  
  
import java.util.function.BiConsumer;  
  
public class BiConsumerExample {  
    public static void main(String[] args) {  
        BiConsumer<String, Integer> student = (name, marks) ->  
            System.out.println(name + " scored " + marks);  
  
        student.accept("Rutu", 95);  
        student.accept("Renu", 88);  
    }  
}
```

BiConsumer with Collections (Key-Value Pairs)

```
package com.java8;  
  
import java.util.*;  
  
public class BiConsumerMapExample {  
    public static void main(String[] args) {  
        Map<String, Integer> scores = new HashMap<>();  
        scores.put("Rutu", 95);  
        scores.put("Renu", 88);  
        scores.put("Sana", 76);  
  
        // forEach in Map uses BiConsumer  
        scores.forEach((name, marks) ->  
            System.out.println(name + " -> " + marks));  
    }  
}
```

Key Points

- BiConsumer → takes 2 inputs, performs an action, returns nothing.
- Useful for working with maps (key, value) or when processing two related inputs.

- Can be chained with `andThen()` like `Consumer`:

```
BiConsumer<String, Integer> c1 = (name, marks) -> System.out.print(name + ": " + marks);
```

```
BiConsumer<String, Integer> c2 = (name, marks) -> System.out.println(" (processed)");
```

```
c1.andThen(c2).accept("Rutu", 95);
```

Output:

Rutu: 95 (processed)

5. Supplier<T>

`Supplier` is a predefined functional interface in `java.util.function`.

- Represents a function that:
 - Takes no input
 - Returns a value of type `T`
- Abstract method: `T get();`

Supplier Example

```
package com.java8;

import java.util.*;

public class BiConsumerMapExample {
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();
        scores.put("Rutu", 95);
        scores.put("Renu", 88);
        scores.put("Sana", 76);

        // forEach in Map uses BiConsumer
        scores.forEach((name, marks) ->
            System.out.println(name + " -> " + marks));
    }
}
```

Key Points

- `Supplier` is useful when:
 - You need to generate values on demand (OTP, random numbers, UUID, timestamps).
 - No input is required, only output.
- If no value is returned, it will throw a compile-time error.
- Commonly used in factory patterns or lazy evaluation.

6. Function<T, R>

`Function<T, R>` is a predefined functional interface in `java.util.function`.

- Represents a function that:
 - Takes one input of type `T`
 - Returns one output of type `R`
- Abstract method: `R apply(T t);`

Example

```
package com.java8;

import java.util.function.Function;

//Function Functional Interface
//predefined interface
//has only one abstract method - apply(T r)

//interface Function<R, T>
//{
//R apply(T t);
//}

//T-> input type
//R-> return type

//takes input and returns output

public class FunctionFI {

    public static void main(String[] args) {
        Function<String, Integer> names = n-> n.length();
        System.out.println(names.apply("Rutu"));
        System.out.println(names.apply("Manu"));

        Function<String, String> upper = u -> u.toUpperCase();
        System.out.println(upper.apply("Rutu"));
        System.out.println(upper.apply("Manu"));
    }
}
```

Key Points

- Function → takes input, returns output.
- Commonly used for:
 - Transformations (map() in streams)
 - Conversions (String to Integer, upper/lowercase, etc.)
- Can be chained using:
 - andThen() → execute this function, then another.
 - compose() → execute another function first, then this one.

Example:

```
Function<String, String> addExclamation = s -> s + "!";
```

```
Function<String, String> toUpper = s -> s.toUpperCase();
```

```
System.out.println(addExclamation.andThen(toUpper).apply("Rutu")); // RUTU!
```

7. BiFunction<T, U, R>

BiFunction<T, U, R> is a predefined functional interface in java.util.function.

- Represents a function that:

- Takes **two inputs** (T and U)
- Returns **one output** (R)
- Abstract method: R apply(T t, U u);
- Use case: Combine two values and return a result.

Example

```
package com.java8;

import java.util.function.BiFunction;

public class BiFunctionExample {
    public static void main(String[] args) {
        BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;

        System.out.println(add.apply(10, 20)); // 30
        System.out.println(add.apply(5, 15));  // 20
    }
}
```

Key Points

- **BiFunction** → 2 inputs, 1 output
- Useful when the **result depends on two input values**.
- Can be chained using andThen() to process the result further.

Example of chaining:

```
BiFunction<Integer, Integer, Integer> multiply = (a, b) -> a * b;
```

```
Function<Integer, Integer> add10 = x -> x + 10;
```

```
System.out.println(multiply.andThen(add10).apply(5, 3)); // (5*3)+10 = 25
```

Optional Class

Introduction to Optional Class in Java

Introduced in: Java 8 (java.util package)

Purpose:

Optional<T> is a **container object** which may or may not contain a **non-null value**.

It was mainly introduced to:

- **Avoid NullPointerException (NPE)**
- **Reduce explicit null checks** (if(obj != null))
- **Encourage functional programming style** (using map, filter, ifPresent etc.)

Key Methods:

Method	Description
Optional.of(value)	Returns an Optional containing the non-null value. Throws NullPointerException if value is null.
Optional.ofNullable(value)	Returns an Optional containing the value if it is non-null , otherwise an empty Optional .
Optional.empty()	Returns an empty Optional (no value present).
get()	Returns the contained value if present, else throws NoSuchElementException.
isPresent()	Returns true if value is present, else false.
ifPresent(Consumer)	Executes the given lambda if value is present.
orElse(defaultValue)	Returns the value if present, else returns the given default.
orElseThrow(Supplier)	Returns the value if present, else throws the exception provided.

Example 1

```
package in.ps.java8.Function;

import java.util.Optional;

/*
1. Optional class introduced in java 1.8v
|
2. It is used to avoid NullPointerException
3. Null checks are not required
*/

public class Optional1 {
    public static void main(String[] args) {
        Optional<String> role = Optional.of("Dev");
        //of(T value)
        // creates an optional with the specified value
        // which must be not null

        System.out.println(role);
        //prints the value of the optional variable role

        System.out.println("Designation " + role.get());

        System.out.println("Empty optional instance "+Optional.empty());
        // returns an empty optional instance
    }
}
```

```

String ans1 = "Yes";
String ans2 = null;

System.out.println(Optional.ofNullable(ans1));
System.out.println(Optional.ofNullable(ans2));
// returns an specified value, (not-null) otherwise returns an empty optional

Optional<String> o = Optional.of("virat");
System.out.println(o.isPresent());
if(o.isPresent()) {
    System.out.println(o.get());
}else {
    System.out.println("name not found");
}

Optional<String> o1 = Optional.of("karthik");
o1.ifPresent(s->System.out.println(s.length()));

String n1 = "Ramesh";//null
String n2 = Optional.ofNullable(n1).orElse("Suresh");
System.out.println(n2);

String n3 = null;//null
String n4 = Optional.ofNullable(n3).orElseThrow(IllegalArgumentException::new);
System.out.println(n4);
}
}

```

```
Optional<String> role = Optional.of("Dev");
```

- Creates an Optional object containing "Dev".
- Since "Dev" is non-null, this is safe.
- Prints something like:
Optional[Dev]
- role.get() returns "Dev".
- Output:
Designation Dev

get() should be used **only if you're sure the value is present**, else it throws NoSuchElementException.

```
System.out.println("Empty optional instance " + Optional.empty());
```

- Creates an empty Optional and prints it.
- Output:
Empty optional instance Optional.empty

```
String ans1 = "Yes";
```

```
String ans2 = null;
```

```
System.out.println(Optional.ofNullable(ans1));
```

```
System.out.println(Optional.ofNullable(ans2));
```

- Optional.ofNullable(ans1) → Optional[Yes]
- Optional.ofNullable(ans2) → Optional.empty
- This is safer than Optional.of(...) because it **does not throw an exception if the value is null**.

```
Optional<String> o = Optional.of("virat");
```

```
System.out.println(o.isPresent());
```

```

if(o.isPresent()) {
    System.out.println(o.get());
} else {
    System.out.println("name not found");
}

```

- isPresent() returns true if value exists.
- Since "virat" is present, output will be:
- true
- virat

```

Optional<String> o1 = Optional.of("karthik");
o1.ifPresent(s -> System.out.println(s.length()));

```

- ifPresent(...) executes the lambda if value is present.
- It prints length of "karthik" which is 7.

```
String n1 = "Ramesh"; // not null
```

```
String n2 = Optional.ofNullable(n1).orElse("Suresh");
```

```
System.out.println(n2);
```

- If n1 is non-null, orElse(...) returns it, else returns "Suresh".
- Output: Ramesh

```
String n3 = null;
```

```
String n4 = Optional.ofNullable(n3).orElseThrow(IllegalArgumentException::new);
```

```
System.out.println(n4);
```

- Here n3 is null.
- orElseThrow(...) will throw a new IllegalArgumentException.
- So this line will **throw an exception and stop the program**. n4 is never printed.

Example 2:

```

package in.ps.java8.Function;

import java.util.Optional;

class ps4{
    public String getMockStatusbyId(Integer id) {
        if(id==10) {
            return "cleared";
        }
        else if(id==11) {
            return "not cleared";
        }else {
            return null;
        }
    }
}

```



```

> public Optional<String> getNameById(Integer id){
    String name = null;
    if(id==10) {
        name="Awez";
    }else if(id==11) {
        name="Suresh";
    }
    return Optional.ofNullable(name);
}

public class Optional2 {
> public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("enter the id: ");
    int id = sc.nextInt();

    ps4 p = new ps4();

    // String s1 = p.getMockStatusById(id);
    // System.out.println("you have "+s1.toUpperCase()+" the mock");

    Optional<String> o = p.getNameById(id);
    if(o.isPresent()) {
        System.out.println(o.get()+" welcome to pentagon space");
    }
    else {
        System.out.println("data not found");
    }
}
}

```

- There's a class ps4 that simulates a small data source.
 - The method getMockStatusById(Integer id) returns a String status based on the given id, and null if no matching id is found.
 - The method getNameById(Integer id) returns an Optional<String> instead of a plain String. It sets a name if id is 10 or 11, otherwise it remains null. This name is then wrapped using Optional.ofNullable(name), which means if name is null, it returns Optional.empty() instead of causing an exception.
- In the Optional2 class, inside main, the user is asked to enter an ID via Scanner.
- That ID is passed to getNameById to retrieve a name wrapped in an Optional.
- Then, isPresent() is used to check whether the Optional actually contains a value.
 - If it does, the program prints a welcome message using the get() method to extract the value.
 - If not, it prints "data not found".

This approach avoids null checks explicitly. Instead of checking if(name != null), the code relies on Optional's isPresent() and get() methods, making the logic safer and more expressive.

Collectors

Example:

```
import java.util.Arrays;
import java.util.List;

class Employee {
    String name;
    double salary;
    String department;

    Employee(String name, double salary, String department) {
        this.name = name;
        this.salary = salary;
        this.department = department;
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }
    public String getDepartment() { return department; }

    @Override
    public String toString() {
        return name + " (" + department + ", " + salary + ")";
    }
}

public class Emp{
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Awez", 30000, "HR"),
            new Employee("Divya", 25000, "Sales"),
            new Employee("Vivek", 35000, "HR"),
            new Employee("Kiran", 28000, "IT"),
            new Employee("Arun", 40000, "Sales")
        );
    }
}
```

Collectors.toList()

toList() collects all elements from a stream into a List. It preserves the order of elements and allows duplicates. This is useful when you want to transform a stream of objects or values and then store them back into a list for further use.

```
import java.util.*;
import java.util.stream.*;

public class ToListExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Awez", 30000, "HR"),
            new Employee("Divya", 25000, "Sales"),
            new Employee("Vivek", 35000, "HR"),
            new Employee("Kiran", 28000, "IT"),
            new Employee("Arun", 40000, "Sales")
        );

        List<String> names = employees.stream()
            .map(Employee::getName)
            .collect(Collectors.toList());

        System.out.println("Employee names: " + names);
    }
}
```

Collectors.groupingBy()

groupingBy() groups elements of a stream based on a classifier function (like department, category, or gender). It returns a Map where each key is a group, and the value is a list of elements in that group. It is powerful for data classification.

```
import java.util.*;
import java.util.stream.*;

public class GroupingByExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Awez", 30000, "HR"),
            new Employee("Divya", 25000, "Sales"),
            new Employee("Vivek", 35000, "HR"),
            new Employee("Kiran", 28000, "IT"),
            new Employee("Arun", 40000, "Sales")
        );

        Map<String, List<Employee>> grouped = employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment));

        System.out.println("Employees grouped by department: " + grouped);
    }
}
```

Collectors.partitioningBy()

partitioningBy() splits elements of a stream into two groups based on a boolean condition (true or false). It returns a Map<Boolean, List<T>>. This is useful when you want to separate elements that match a condition from those that do not.

```
import java.util.*;
import java.util.stream.*;

public class PartitioningByExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Awez", 30000, "HR"),
            new Employee("Divya", 25000, "Sales"),
            new Employee("Vivek", 35000, "HR"),
            new Employee("Kiran", 28000, "IT"),
            new Employee("Arun", 40000, "Sales")
        );

        Map<Boolean, List<Employee>> partitioned = employees.stream()
            .collect(Collectors.partitioningBy(e -> e.getSalary() > 30000));

        System.out.println("Salary > 30000? " + partitioned);
    }
}
```

Collectors.summarizingDouble() (→ DoubleSummaryStatistics)

summarizingDouble() processes a stream of elements and returns a DoubleSummaryStatistics object containing count, sum, average, min, and max in one go. It is the easiest way to get all key statistics about numeric data from a stream.

```
import java.util.*;
import java.util.stream.*;
import java.util.DoubleSummaryStatistics;

public class SummarizingDoubleExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("Awez", 30000, "HR"),
            new Employee("Divya", 25000, "Sales"),
            new Employee("Vivek", 35000, "HR"),
            new Employee("Kiran", 28000, "IT"),
            new Employee("Arun", 40000, "Sales")
        );

        DoubleSummaryStatistics stats = employees.stream()
            .collect(Collectors.summarizingDouble(Employee::getSalary));

        System.out.println("Count = " + stats.getCount());
        System.out.println("Sum = " + stats.getSum());
        System.out.println("Average = " + stats.getAverage());
        System.out.println("Min = " + stats.getMin());
        System.out.println("Max = " + stats.getMax());
    }
}
```

Introduction to Java Stream API

The **Stream API** in Java (introduced in Java 8) is a powerful feature used to **process data in a functional and declarative way**. While **collections** (like List, Set, Map) are used to **store data**, the Stream API is used to **process or manipulate data** stored in these collections or arrays. A **Stream** represents a **sequence of elements** coming from a source (collection/array), supporting various **aggregate operations** such as filter(), map(), sorted(), forEach(), collect(), etc.

Important: Streams do not change or modify the original data structure. They only process and return results based on the data source.

Key Characteristics of Streams

- **Not a data structure:** Streams don't store elements; they just process them.
- **Source-based:** Streams take input from Collections, Arrays, or I/O channels.
- **Functional in nature:** Use lambda expressions to operate on data.
- **Lazy evaluation:** Intermediate operations are not executed until a terminal operation is called.
- **Single-use:** Once consumed, a stream cannot be reused.
- **Pipeline operations:** Multiple operations can be chained together to form a pipeline.

Stream Creation

There are **two main ways to create a stream**:

1. **Using Stream.of()**
 - This is a **static method of the Stream interface**.
 - Useful for creating a stream from a fixed set of elements.
2. `Stream<Integer> st = Stream.of(7, 4, 9, 5);`
3. `st.sorted().forEach(System.out::println);`
4. **Using stream() method on a Collection**
 - Every collection class (List, Set, etc.) has a stream() method.
 - It converts the collection into a stream.
5. `ArrayList<String> al = new ArrayList<>();`
6. `al.add("eena");`
7. `al.add("meena");`
8. `al.add("deeka");`
9. `Stream<String> st2 = al.stream();`
10. `st2.sorted().forEach(System.out::println);`

Example:

```
package in.ps.java8.StreamAPI;

import java.util.ArrayList;

/*
Stream API used to process the data.
Collection are used to store to data.
It can be defined as sequence of elements that comes from a source
Source of data can be Array or collection.
Stream is not a data structure
Stream means bunch of operation applied on the source data
Stream will not change the original data structure of the source
(it will just process the data given by the source)

Stream creation: in 2 ways
1. Stream.of(e1,e2,e3...)
   //Interface

2. stream() method
   //Method in the collection(list,set,map..)
*/

public class Demo1 {
    public static void main(String[] args) {

        //1st way: Stream.of()
        Stream<Integer> st = Stream.of(7,4,9,5);
        st.forEach(System.out::println);
        st.sorted().forEach(System.out::println);

        //2nd way: stream()
        ArrayList<String> al = new ArrayList<>();
        al.add("eena");
        al.add("meena");
        al.add("deeka");
        Stream<String> st2 = al.stream();
        st2.forEach(System.out::println);
        st2.sorted().forEach(System.out::println);
    }
}
```

This program demonstrates how to **create and use Java Streams** using both methods.

- The program defines a class Demo1 with a main method.
- In the **first approach**, it uses Stream.of() to create a stream of integers 7, 4, 9, 5.
 - The stream is then processed using terminal operations such as forEach() and sorted().
- In the **second approach**, it creates an ArrayList<String> with three elements — "eena", "meena", and "deeka".
 - It calls stream() on the list to get a stream and processes it using forEach() and sorted() to print the names in sorted order.
- Stream.of() is used for direct stream creation from values.
- stream() is used to create a stream from a collection.
- forEach(System.out::println) is a **terminal operation** used to print each element.
- sorted() is an **intermediate operation** used to sort stream elements.
- Streams are **single-use** — once a terminal operation is called, the stream is consumed.
- The original collection remains **unchanged** even after processing.

Example

```
public class Lambda6 {
    public static void main(String[] args) {
        //Runnable r = new Runnable(); //cannot create an object of the interface

        //without lambda
        Runnable r = new Runnable() {

            @Override
            public void run() {
                System.out.println("ee sala cup namde");
            }

        };
        new Thread(r).start();

        //with lambda
        Runnable r1 = ()-> System.out.println("ee sala cup namdu");
        new Thread(r1).start();

        //using lambda expression to create a new thread
        new Thread(()->{
            for(int i=0; i<5; i++) {
                System.out.println("thread "+i);
            }
        }).start();
    }
}
```

- This program demonstrates creating and running threads using the Runnable interface.
- It first shows the traditional approach using an anonymous inner class to implement Runnable.
- The run() method inside it prints "ee sala cup namde" when the thread runs.
- A new thread is created by passing this Runnable object to the Thread constructor and started using start().
- Next, it shows the same logic implemented using a lambda expression because Runnable is a functional interface with a single abstract method.
- This lambda-based approach is shorter and avoids the boilerplate code of an anonymous class.
- Finally, it directly creates and starts a thread using a lambda expression inline, which runs a loop printing "thread i" five times.
- This illustrates how lambda expressions make thread creation more concise, readable, and efficient when using functional interfaces like Runnable.

Filtering with Streams:

Filtering is one of the most common operations in the Java Stream API.

It is used to **select only the elements from a collection that match a given condition** (predicate), and **discard all others**. This makes it easier to process only the data you care about.

```
package in.ps.java8.StreamAPI;

import java.util.Arrays;

public class Demo2 {
    public static void main(String[] args) {
        List<Integer> al = Arrays.asList(10,20,6,16);

        al.stream().filter(k->k>10).forEach(System.out::println);
    }
}
```

Arrays.asList(...): Creates a fixed-size list containing the given elements.

stream(): Converts the list into a stream, allowing us to use Stream API operations.

filter(Predicate)

- An **intermediate operation** that takes a Predicate (boolean condition) as input.
- Returns a new stream consisting of only those elements that satisfy the condition.

Lambda Expression $k \rightarrow k > 10$

- For each element k , checks if it is greater than 10.
- This is a functional style approach introduced in Java 8.

forEach(System.out::println)

- A **terminal operation** that consumes the filtered elements and prints them.

Mapping operation:

It is used to **transform the stream elements** and **return the transformed elements** as new stream.

EX: Stream map (Function function)

Flat map function

It is used to flatten list of streams into single stream

```
package com.StreamAPI;

import java.util.Arrays;

//What is flatMap(Function f) method ?
//It is used to flatten list of streams into single stream.

public class StreamAPI3 {
    public static void main(String[] args) {
        System.out.println("Before using flat map");
        List<String> languages = Arrays.asList("C#", "java", "C++", "scala");
        List<String> ftech = Arrays.asList("HTML", "CSS", "Angular", "ReactJS");
        List<List<String>> developer = Arrays.asList(languages, ftech);
        developer.stream().forEach(System.out::println);

        System.out.println();

        System.out.println("after using flat map");
        Stream<String> s1=developer.stream().flatMap(s->s.stream());
        s1.forEach(System.out::println);
    }
}
```

Slicing Operations with Stream

distinct(): to get **unique element** from Stream.

limit(long maxSize): **Get elements** from the stream based on **given size**.

skip(long n): it is used to **skip given number of elements** from **starting position** of the stream.

```

1 package com.StreamAPI;
2
3 import java.util.Arrays;
4
5
6 public class StreamAPI4 {
7     public static void main(String[] args) {
8         List<String> lang = Arrays.asList("C#", "java", "C++", "scala");
9
10        lang.stream().limit(2).forEach(System.out::println);
11        System.out.println("-----");
12
13        lang.stream().skip(2).forEach(System.out::println);
14        System.out.println("-----");
15
16        List<String> names = Arrays.asList("kiran", "varun", "varun", "varun", "madhu", "kiran");
17        names.stream().distinct().forEach(System.out::println);
18    }
19 }
20 }
21

```

Matching Operations with Stream:

1. boolean anyMatch(Predicate p)
2. boolean allMatch(Predicate p)
3. boolean noneMatch(Predicate p)

```

import java.util.Arrays;

class Book{
    String authorName;
    String country;

    public Book(String authorName, String country) {
        super();
        this.authorName = authorName;
        this.country = country;
    }
}

public class StreamAPI5 {
    public static void main(String[] args) {
        Book b1=new Book("kartgik","India");
        Book b2=new Book("kiran","Aus");
        Book b3=new Book("varun","UK");
        Book b4=new Book("vidya","US");

        List<Book> list=Arrays.asList(b1,b2,b3,b4);

        boolean b5 = list.stream().anyMatch(k -> k.country.equals("India"));
        System.out.println(b5);

        boolean b6 = list.stream().allMatch(k -> k.country.equals("UK"));
        System.out.println(b6);

        boolean b7 = list.stream().noneMatch(k -> k.country.equals("Sri Lanka"));
        System.out.println(b7);
    }
}

```


Converting Entity to DTO using Java Streams

```
import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

class TCS {
    private String id;
    private String name;
    private Double salary;

    public TCS(String id, String name, Double salary) {
        super();
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Double getSalary() {
        return salary;
    }

    @Override
    public String toString() {
        return "TCS [id=" + id + ", name=" + name + ", salary=" + salary + "]";
    }
}

class TCSDTO {
    private String id;
    private String name;
    private Double salary;

    public TCSDTO(String id, String name, Double salary) {
        super();
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Double getSalary() {
        return salary;
    }
}

public class Demo2 {

    public static void main(String[] args) {

        Set<TCS> emp = new HashSet<>();
        emp.add(new TCS("TCS10", "Karthik", 23444.55));
        emp.add(new TCS("TCS11", "Kiran", 24444.55));
        emp.add(new TCS("TCS12", "Divya", 25444.55));

        System.out.println("SET ref = " + emp);

        // Convert Entity to DTO using Stream API
        Set<TCSDTO> empdto = emp.stream()
            .map(e -> new TCSDTO(e.getId(), e.getName(), e.getSalary()))
            .collect(Collectors.toSet());

        // Display DTOs
        empdto.forEach(dto -> System.out.println(dto.getId() + " - " + dto.getName() +
            " - " + dto.getSalary()));
    }
}
```

1. Entity Class (TCS)

- Represents an employee with id, name, and salary.
- Provides getter methods to access its fields.

2. DTO Class (TCSDTO)

- Used for transferring employee data without exposing the entity.
- Contains the same fields but only for data transfer purposes.

3. Creating a Set of Employees

- A `HashSet<TCS>` is created and populated with sample employee data.

4. Converting Entity to DTO

- `emp.stream()` creates a stream from the `HashSet`.
- `.map(e -> new TCSDTO(e.getId(), e.getName(), e.getSalary()))` converts each TCS object to a TCSDTO.
- `.collect(Collectors.toSet())` collects all DTOs into a `Set<TCSDTO>`.

5. Displaying DTOs

- The DTO set is iterated using `forEach`.
- Prints the id, name, and salary of each DTO.

Interview Question 1

```
1 package com.StreamAPI;
2
3 import java.util.ArrayList;
4
5 class Employee {
6     int id;
7     String name;
8     int age;
9     String gender;
10    String department;
11    int yearOfJoining;
12    double salary;
13
14    public Employee(int id, String name, int age, String gender, String department, int yearOfJoining, double salary) {
15        super();
16        this.id = id;
17        this.name = name;
18        this.age = age;
19        this.gender = gender;
20        this.department = department;
21        this.yearOfJoining = yearOfJoining;
22        this.salary = salary;
23    }
24
25    public int getId() {
26        return id;
27    }
28
29    public void setId(int id) {
30        this.id = id;
31    }
32 }
```

```

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
    this.gender = gender;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public int getYearOfJoining() {
    return yearOfJoining;
}

public void setYearOfJoining(int yearOfJoining) {
    this.yearOfJoining = yearOfJoining;
}

public double getSalary() {
    return salary;
}

public void setSalary(double salary) {
    this.salary = salary;
}

@Override
public String toString() {
    return "Employee [id=" + id + ", name=" + name + ", age=" + age + ", gender=" + gender + ", department="
        + department + ", yearOfJoining=" + yearOfJoining + ", salary=" + salary + "];"
}

```

```

public class StreamAPI8 {
    public static void main(String[] args) {
        List<Employee> emplist = new ArrayList<>();
        emplist.add(new Employee(10, "karthik", 34, "male", "developer", 2012, 30000.0));
        emplist.add(new Employee(11, "divya", 28, "female", "HR", 2015, 25000.0));
        emplist.add(new Employee(12, "vivek", 30, "male", "sales and marketing", 2015, 14000.0));
        emplist.add(new Employee(13, "kiran", 32, "male", "product development", 2014, 32000.0));
        emplist.add(new Employee(14, "arun", 36, "male", "developer", 2016, 30000.0));
        emplist.add(new Employee(15, "punith", 31, "male", "HR", 2015, 25000.0));
        emplist.add(new Employee(21, "varun", 29, "male", "HR", 2015, 25001.0));
        emplist.add(new Employee(16, "yamuna", 50, "female", "sales and marketing", 2015, 14500.0));
        emplist.add(new Employee(17, "keerthi", 34, "female", "product development", 2014, 33000.0));
    }
}

```

```

// Group By is used to categorize the data / grouping the data
// when we use groupingBy() function with stream it will group the data as key -
// value(S) Pair and it will return to map object
// 1] How many male and female employees are there
Map<String, Long> temp = emplist.stream().collect(Collectors.groupingBy // first we will group by the gender and
                                                                    // then count it both the methods belong
                                                                    // to the collectors class

(Employee::getGender, Collectors.counting()));
System.out.println(temp);

System.out.println();

// 2] Print the name of the department
emplist.stream().map(Employee::getDepartment).distinct().forEach(System.out::println);

System.out.println();

```

```

// 3] Avg mage of male and female of employee
Map<String, Double> avg = emplist.stream()
    .collect(Collectors.groupingBy(Employee::getGender, Collectors.averagingInt(Employee::getAge)));
System.out.println(avg);

System.out.println();

// 4] highest paid employee in organization
Optional<Employee> hp = emplist.stream()
    .collect(Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)));
System.out.println(hp.get().getName());

System.out.println();

// 5] Get the employee who joined after 2015
// Optional <Employee> emp = emplist.stream().filter(p -> p.getYearOfJoining() >
// 2015)
// .map(Employee::getName).forEach(System.out::println);

// 6] count the number of employees in each department
Map<String, Long> deptCount = emplist.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()));
System.out.println(deptCount);

System.out.println();

// 7] Average salary of each department
Map<String, Double> avgSalaryByDept = emplist.stream().collect(
    Collectors.groupingBy(Employee::getDepartment, Collectors.averagingDouble(Employee::getSalary)));
System.out.println(avgSalaryByDept);

System.out.println();

//8.get the young emp of hr dept male
Optional<Employee1> e4=emplist.stream().filter(p->p.getGender()=="male"&&
p.getDepartment()=="HR").min(Comparator.comparingInt(Employee1::getAge));
System.out.println(e4);
System.out.println("-----");

//9. max exp in org
Optional<Employee1> e5= emplist.stream().collect(Collectors.minBy
(Comparator.comparing(Employee1::getYearOfJoining)));
System.out.println(e5);
//or
Optional<Employee1> e6= emplist.stream().sorted(Comparator.comparingInt
(Employee1::getYearOfJoining)).findFirst();
System.out.println(e6);
Employee1 e=e6.get();
System.out.println(e);
//
System.out.println("id="+e.getId());
System.out.println("name"+e.getName());
System.out.println("-----");

//10. hoe many males and females present in hr dept
Map<String, Long> e7=emplist.stream().filter(p->p.getDepartment()=="HR")
    .collect(Collectors.groupingBy(Employee1::getGender,Collectors.counting()));
System.out.println(e7);
System.out.println("-----");

//11. what is the avg salary of male and female
Map<String,Double> e8=emplist.stream().collect(Collectors.groupingBy
(Employee1::getGender,Collectors.averagingDouble(Employee1::getSalary)));
System.out.println(e8);
System.out.println("-----");

//12.List of all emp in each dept
Map<String,List<Employee1>>e9=emplist.stream().collect(Collectors.groupingBy
(Employee1::getDepartment));
System.out.println(e9);
System.out.println("-----");

```

```

//13. AVg sal and total sal of whole organization
DoubleSummaryStatistics d= emplist.stream().collect(Collectors.summarizingDouble
(Employee1::getSalary));
//process the stream and calculates statistics based on salary
System.out.println("count= " +d.getCount());
System.out.println("avg= "+d.getAverage());
System.out.println("total sum= "+d.getSum());
System.out.println("Min= "+d.getMin());
System.out.println("Max= "+d.getMax());
System.out.println("-----");
//14.Dept with the highest number of employee
Map<String , Long> dc=emplist.stream().collect(Collectors.groupingBy
(Employee1::getDepartment,Collectors.counting()));
Optional<Map.Entry<String, Long>> maxDept=dc.entrySet().stream().max
(Map.Entry.comparingByValue());
//ComparingByValue()--gives you a comparator that compares the entries
//based on the value
if(maxDept.isPresent()) {
    System.out.println("Dept:" +maxDept.get().getKey()+"-"+ maxDept.get()
        .getValue()+" employees");
}
else {
    System.out.println("no departments found");
}
System.out.println("-----");
emplist.stream().collect(Collectors.groupingBy(Employee1::getDepartment,
Collectors.counting()))).entrySet().stream().max(Map.Entry.comparingByValue())
//Returns an optional <Map.Entry<String,Long>>
//finds the max value
.ifPresent(e1->System.out.println("Department name and employess count: "
+e1.getKey()+" dept "+e1.getValue()+" employees"));
System.out.println("-----");

//15. Second Highest Salary of Employee
Double sh= emplist.stream().map(Employee1::getSalary).distinct().sorted
(Comparator.reverseOrder()).skip(1).findFirst().orElse(0.0);
System.out.println(sh);
System.out.println("-----");
}

```

Interview Question 2

```

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

// Product class
class Product {
    private int id;
    private String name;
    private Double cost;

    public Product(int id, String name, Double cost) {
        this.id = id;
        this.name = name;
        this.cost = cost;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public Double getCost() { return cost; }
}

// Main class
public class Demo2 {

    public static void main(String[] args) {

        List<Product> list = new ArrayList<>();
        list.add(new Product(1, "Java", 28500.5));
        list.add(new Product(2, "Python", 27500.5));
        list.add(new Product(3, "Mern", 26500.5));
        list.add(new Product(4, "Testing", 25500.5));
        list.add(new Product(5, "Mean", 30000.0));
    }
}

```



```

// 1. Display product name and cost where cost > 27k
List<String> list1 = list.stream()
    .filter(p -> p.getCost() > 27000)
    .map(p -> p.getCost() + " " + p.getName())
    .collect(Collectors.toList());
list1.forEach(System.out::println);

System.out.println("-----");

// 2. Display product name where cost = 30k
list.stream()
    .filter(p -> p.getCost() == 30000)
    .forEach(p -> System.out.println(p.getName()));

System.out.println("-----");

// 3. Total cost of all products
System.out.println("Total cost = " + list.stream()
    .collect(Collectors.summingDouble(Product::getCost)));

System.out.println("-----");

// 4. Max and min cost of the product
System.out.println("Max cost = " + list.stream()
    .max(Comparator.comparingDouble(Product::getCost))
    .get().getName());

System.out.println("Min cost = " + list.stream()
    .min(Comparator.comparingDouble(Product::getCost))
    .get().getName());

System.out.println("-----");

// 5. Convert List to Set (cost < 30k)
System.out.println("List to Set = " + list.stream()
    .filter(p -> p.getCost() < 30000)
    .map(Product::getCost)
    .collect(Collectors.toSet()));

System.out.println("-----");

// 6. Convert List to Map
Map<Integer, String> map = list.stream()
    .collect(Collectors.toMap(Product::getId, Product::getName));
System.out.println(map);
}

```

Product Class

- Represents a product with id, name, and cost.
- Provides getters for all fields.

Creating Product List

- A List<Product> is created and populated with 5 products.

Filter and Map (Cost > 27k)

- list.stream() creates a stream of products.
- .filter(p -> p.getCost() > 27000) filters products with cost greater than 27,000.
- .map(p -> p.getCost() + " " + p.getName()) maps products to a string showing cost and name.
- .collect(Collectors.toList()) collects the results into a list.
- Prints all products satisfying the condition.

Filter products with Cost = 30k

- Filters the product with cost exactly 30,000.
- Prints the name of the product.

Total Cost of All Products

- Uses `Collectors.summingDouble(Product::getCost)` to calculate the total cost of all products.
- Prints the total cost.

Maximum and Minimum Cost Products

- `max(Comparator.comparingDouble(Product::getCost))` finds the product with the highest cost.
- `min(Comparator.comparingDouble(Product::getCost))` finds the product with the lowest cost.
- Prints the names of the products with max and min costs.

Convert List to Set (Cost < 30k)

- Filters products with cost < 30,000.
- Maps to product costs.
- Collects into a Set to remove duplicates.

Convert List to Map

- Uses `Collectors.toMap(Product::getId, Product::getName)` to convert the list into a map with id as key and name as value.
- Prints the resulting map.

Interview Question 3

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

// Student class
class Student {
    String name;
    String college;

    public Student(String name, String college) {
        this.name = name;
        this.college = college;
    }

    @Override
    public String toString() {
        return "Student [name=" + name + ", college=" + college + "]";
    }
}

// Main class
public class Demo1 {

    public static void main(String[] args) {

        List<Student> list = Arrays.asList(
            new Student("karthik", "RVCE"),
            new Student("divya", "RVCE"),
            new Student("kiran", "PES"),
            new Student("varun", "BMS"),
            new Student("harish", "NIE")
        );

        // Filter students from RVCE
        List<Student> rvceStudents = list.stream()
            .filter(s -> s.college.equals("RVCE"))
            .collect(Collectors.toList());

        // Print student names
        rvceStudents.forEach(s -> System.out.println(s.name));
    }
}
```

- **Student Class**
 - Represents a student with name and college.
 - Overrides toString() for readable output.
- **Creating Student List**
 - Uses Arrays.asList() to create a list of 5 students from different colleges.
- **Filter Students by College**
 - list.stream() creates a stream of students.
 - .filter(s -> s.college.equals("RVCE")) filters students whose college is "RVCE".
 - .collect(Collectors.toList()) collects them into a list.
- **Display Names of Filtered Students**
 - Iterates over the filtered list using forEach and prints the name of each student.